

实验一：手写数字识别

基于卷积神经网络的 MNIST 分类

尧祥临 202518023406038

中国科学院大学 前沿交叉科学学院
2026 年春季学期 深度学习

实验任务

目标

在 MNIST 手写数字数据集上构建并训练卷积神经网络，实现 0 到 9 共 10 类数字图象分类。

实验要求

- 使用 PyTorch 完成数据加载、模型构建与训练
- 搭建规范的 CNN 网络结构
- 测试集准确率达到 98% 及以上

本次完成情况

- 训练 20 个 epoch
- 保存训练日志和模型参数
- 最终测试集准确率：**99.33%**

数据集与划分

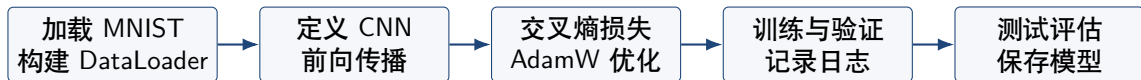
- 数据集: MNIST 手写数字数据集
- 输入尺寸: $1 \times 28 \times 28$ 灰度图象
- 类别数: 10 类, 对应数字 0 到 9
- 输入预处理: `transforms.ToTensor()`

划分	样本数
训练集	55000
验证集	5000
测试集	10000

划分方式

从原始 60000 张训练图象中随机划分 55000 张用于训练, 5000 张用于验证; 测试集保持官方 10000 张不变。

整体流程



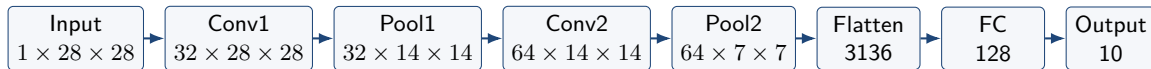
训练阶段

- `model.train()`
- 前向传播、反向传播、参数更新

评估阶段

- `model.eval()`
- 计算 loss 和 accuracy

CNN 模型结构



卷积与池化

两个卷积层均采用 5×5 卷积核、stride 1、padding 2；最大池化采用 2×2 、stride 2。

分类头

将 $64 \times 7 \times 7$ 特征图展平为 3136 维向量，经过 128 维隐藏层后输出 10 类 logits。

模型核心代码

```
class CNN(nn.Module):
    def __init__(self, hidden_dim=128):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=5, stride=1, padding=2)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.conv2 = nn.Conv2d(32, 64, kernel_size=5, stride=1, padding=2)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 10)
        )

    def forward(self, x):
        x = self.pool1(self.relu1(self.conv1(x)))
        x = self.pool2(self.relu2(self.conv2(x)))
        return self.fc(x)
```

参数量与尺寸变化

层	输出尺寸	参数量
Conv1	$32 \times 28 \times 28$	832
Conv2	$64 \times 14 \times 14$	51264
Linear1	128	401536
Linear2	10	1290
总计	—	454922

尺寸计算

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - K}{S} \right\rfloor + 1$$

- padding 2 保持卷积前后尺寸不变
- 两次最大池化将 $28 \rightarrow 14 \rightarrow 7$
- 参数主要集中在第一个全连接层

项目	设置
Batch size	64
Epochs	20
Learning rate	5×10^{-4}
隐藏层维度	128
优化器	AdamW
损失函数	CrossEntropyLoss
训练设备	CUDA GPU

损失函数

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(z_{i,y_i})}{\sum_{c=0}^9 \exp(z_{i,c})}$$

- 每个 epoch 后记录验证集和测试集指标
- 训练日志保存为 `train_log.csv`
- 模型参数保存为 `cnn_mnist.pth`

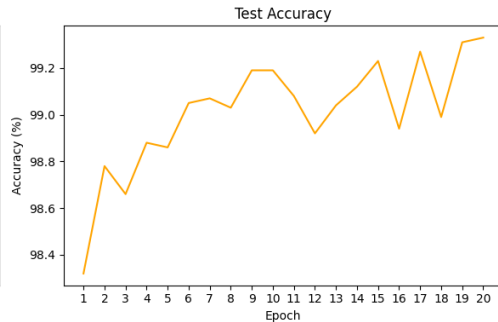
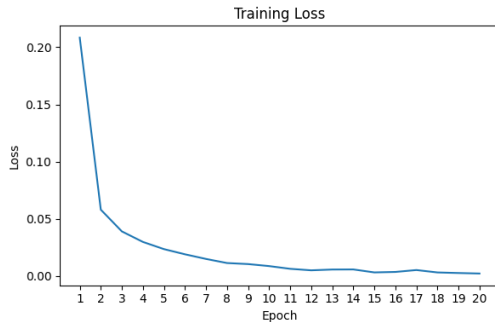
训练结果

Epoch	Train Loss	Val Loss	Val Acc	Test Loss	Test Acc
1	0.2084	0.0680	97.78%	0.0518	98.32%
5	0.0235	0.0384	98.80%	0.0321	98.86%
10	0.0087	0.0326	99.18%	0.0308	99.19%
15	0.0031	0.0319	99.14%	0.0297	99.23%
19	0.0026	0.0267	99.32%	0.0310	99.31%
20	0.0022	0.0289	99.20%	0.0312	99.33%

观察

第 1 个 epoch 后测试准确率已经达到 98.32%，超过实验要求；训练到第 20 个 epoch 后测试准确率提升到 99.33%。

训练曲线



- 训练损失在前几个 epoch 快速下降，随后逐渐收敛
- 测试准确率整体位于 99% 附近并继续提升
- 第 16 个 epoch 附近存在一次波动，但后续恢复到较低 loss 和较高 accuracy

结果分析

模型有效性

两层卷积能够提取局部笔画、边缘和组合模式；对 MNIST 这种规整的灰度分类任务，浅层 CNN 已经具备较强表达能力。

- 最终测试准确率：**99.33%**
- 实验要求：测试准确率不低于 98%
- 模型文件：Lab1/model/cnn_mnist.pth

泛化表现

训练后期验证集和测试集准确率保持在 99% 左右，没有出现持续恶化的过拟合趋势。

总结

完成内容

本次实验基于 PyTorch 完成了 MNIST 手写数字识别，包括数据加载与划分、CNN 模型搭建、训练评估、日志记录、模型保存和结果可视化。

主要收获

进一步熟悉了卷积、激活、池化和全连接分类头的配合方式，也掌握了 Dataset、DataLoader、nn.Module、反向传播和优化器更新的基本流程。

可改进方向

固定随机种子增强复现性；加入输入标准化；补充混淆矩阵，分析不同数字类别之间的错误模式。